🧠 BASICS OF C#

1️⃣ WHAT IS C# REALLY DOING?

A program is simply:

Instructions that manipulate data stored in memory.

Every C# program answers three questions:

1. **Where is data stored?** → Variables
2. **What type of data?** → Data Types
3. **How decisions are made?** → Control Structures

How C# Executes

C# Code

↓

Compiler (csc / Roslyn)

↓`

Intermediate Language (IL)

↓

CLR (Common Language Runtime)

↓

Machine Code (JIT)

CLR manages:

- Memory
- Type safety
- Exception handling
- Garbage collection

Think of CLR as a **smart operating manager** supervising your program.

2️⃣ VARIABLES

A **variable** is a **named memory container** used to store data.

Real-life analogy:

Imagine a college office:

Label	Cupboard	Content

studentName	Drawer A	"Naveen"
marks	Drawer B	90

Variable name = label
Memory = cupboard
Value = stored item

Syntax

datatype variableName = value;
Example:
int age = 23;
string name = "Naveen";

What Happens Internally?

When you write:
int age = 23;
Steps:

1. Compiler reserves **4 bytes memory**
2. Labels memory as age
3. Stores value 23

Variable Naming Rules

✅ Allowed:
studentName
totalMarks
_age
❌ Not allowed:
1name
class
int

Case Sensitivity

int age = 20;
int Age = 30;
These are **different variables**.

Variable Types by Scope

1. Local Variable

Inside method.

```
void Show()
{
    int x = 10;
}
```

Exists only inside function.

2. Instance Variable

Inside class.

```
class Student
{
    int marks;
}
```

Each object gets its own copy.

3. Static Variable

Shared across objects.

```
static int count;
```

Used for counters.

Real-life example:

- Total students in college.

var Keyword (Important)

```
var city = "Bhimavaram";
```

Compiler automatically decides datatype.

⚠️ NOT dynamic typing.

Type fixed at compile time.

Constants

Values that never change.

```
const double PI = 3.14;
```

Real-life:

- Aadhaar number
- Mathematical constants

3️⃣ DATA TYPES

Data type defines:

✅ Memory size

✅ Allowed operations

✅ Range of values

Two Main Categories

Data Types

├ Value Types

└── Reference Types

VALUE TYPES

Stored in **STACK memory**.

Contain actual value.

Common Value Types

Type	Example	Size	Use
int	10	4 bytes	counting
double	10.5	8 bytes	decimals
char	'A'	2 bytes	single letter
bool	true	1 byte	decisions
decimal	money	16 bytes	finance

Example

```
int marks = 95;
bool passed = true;
```

Stack Memory Concept

Stack works like plates stacked vertically.

| marks = 95 |

| passed=true|

Fast allocation & deletion.

REFERENCE TYPES

Stored in **HEAP memory**.

Variable stores **address**, not value.

Example:
string name = "Naveen";
Memory:
Stack → address → Heap → "Naveen"

Why Reference Types?

Efficient for large objects.

Example:

- Images
- Files
- Objects

Built-in Data Types

Numeric Types

int students = 60;

double temperature = 36.5;

decimal salary = 50000.75m;

Use decimal for money (high precision).

Character Type

char grade = 'A';

Stores Unicode character.

Boolean Type

bool isLoggedIn = true;

Used in decision making.

String Type

string college = "Vishnu College";

Immutable (cannot change internally).

Type Conversion

Implicit Conversion (Safe)

int x = 10;

double y = x;

Small → Large.

Explicit Conversion (Casting)

double d = 10.5;

```
int x = (int)d;
```
Data loss possible.

Parsing

```
string num = "100";
int value = int.Parse(num);
```
Real-life:

User input from textbox.

TryParse (Safe Method)

```
int result;
bool ok = int.TryParse("123", out result);
```
Prevents crashes.

4 CONTROL STRUCTURES

Control structures decide **program flow**.

Without them, programs behave like a robot repeating same action.

Decision Making Structures

IF Statement

Real-life example:

If student marks ≥ 40 → Pass

```
int marks = 75;
if(marks >= 40)
{
    Console.WriteLine("Pass");
}
```

IF–ELSE

```
if(marks >= 40)
{
    Console.WriteLine("Pass");
}
else
{
```

```
    Console.WriteLine("Fail");
}
```

ELSE IF Ladder

Used for grading system.

```
if(marks >= 90)
    Console.WriteLine("A");
else if(marks >= 75)
    Console.WriteLine("B");
else
    Console.WriteLine("C");
```

Real-Life Scenario

ATM Machine:
IF PIN correct
    Allow transaction
ELSE
    Show error

SWITCH STATEMENT

Best when checking multiple fixed values.

```
int day = 2;
switch(day)
{
    case 1:
    Console.WriteLine("Monday");
    break;
    case 2:
    Console.WriteLine("Tuesday");
    break;
    default:
    Console.WriteLine("Invalid");
    break;
}
```

Why switch is faster?
Compiler creates jump table internally instead of multiple comparisons.

LOOPING STRUCTURES

Used for repetition.

FOR LOOP

Known iterations.

```
for(int i=1;i<=5;i++)
{
    Console.WriteLine(i);
}
```

Real-life:
Attendance numbers.

WHILE LOOP

Unknown repetitions.

```
int i = 1;
while(i <= 5)
{
    Console.WriteLine(i);
    i++;
}
```

Example:
Keep asking password until correct.

DO-WHILE LOOP

Runs at least once.

```
do
{
    Console.WriteLine("Enter choice");
} while(false);
```

ATM menu example.

FOREACH LOOP

Used for collections.

```
int[] marks = {90,80,70};
foreach(int m in marks)
{
        Console.WriteLine(m);
}
```

CONTROL KEYWORDS

break

Stops loop immediately.

```
if(i==3)
        break;
```

continue

Skips iteration.

```
if(i==3)
        continue;
```

return

Exits method.

COMMON MISTAKES

❌ **Using = instead of ==**

```
if(x = 5) // WRONG
```

Correct:

```
if(x == 5)
```

❌ **Infinite Loop**

```
while(true)
{
}
```

CPU stuck.

❌ **Wrong datatype**

```
int price = 10.5; // error
```

Use double.

REAL WORLD MINI PROGRAM

Student Result System:

```
using System;
class Program
{
      static void Main()
   {
      Console.Write("Enter Marks: ");
      int marks = int.Parse(Console.ReadLine());
      if(marks >= 90)
      Console.WriteLine("Grade A");
      else if(marks >= 75)
      Console.WriteLine("Grade B");
      else if(marks >= 40)
      Console.WriteLine("Pass");
      else
      Console.WriteLine("Fail");
   }
}
```

1️⃣ WHY OBJECT-ORIENTED PROGRAMMING EXISTS

Before OOP, programs were written as long procedural code:

Step1 → Step2 → Step3 → Step4

Problems:

- Hard to maintain
- Code duplication
- Difficult teamwork
- Bugs spread easily

OOP solves this by modeling **real-world entities**.

Real-Life Analogy

Think about a **college system**:

Real World	Programming
Student	Class
Individual student	Object
Teacher teaches	Method
Student data	Properties

Programming becomes a **digital representation of reality**.

2 CLASS

A **class** is a blueprint used to create objects.

Blueprint ≠ Building

Class ≠ Object

Example

Blueprint of Student:

```
class Student
{
      public string Name;
      public int Age;

      public void Study()
   {
      Console.WriteLine("Student studying");
   }
}
```

Creating Object

```
Student s1 = new Student();
s1.Name = "Naveen";
s1.Study();
```

Memory Representation

Stack Heap

---

s1 --------> Student Object

Name = Naveen

Age = 23

Object lives in **heap memory**.

Class Components

Component	Purpose
Fields	Store data
Properties	Controlled access
Methods	Behavior
Constructors	Initialization

Constructor

Automatically runs when object created.

```
class Student
{
    public string Name;
    public Student(string name)
  {
    Name = name;
  }
}
```

Usage:

Student s = new Student("Naveen");

Why Constructor?

Real-life:

When a student joins college → registration happens immediately.

3️⃣ ENCAPSULATION (DATA PROTECTION)

Hide internal data using properties.

```
class BankAccount
{
      private double balance;
      public double Balance
   {
      get { return balance; }
      set
      {
      if(value >= 0)
            balance = value;
      }
   }
}
```

Students cannot set negative balance.

Auto Properties

```
public string Name { get; set; }
```

Compiler creates hidden field automatically.

4️⃣ INHERITANCE

A class can reuse another class features.

Child inherits Parent behavior.

Real-Life Example

Person
├Student
└── Teacher

Both share:

- Name
- Age
- Speak()

Code Example

```
class Person
{
    public string Name;
    public void Speak()
  {
    Console.WriteLine("Speaking");
  }
}
class Student : Person
{
    public void Study()
  {
    Console.WriteLine("Studying");
  }
}
```

Usage:

```
Student s = new Student();
s.Speak(); // inherited
```

Types of Inheritance in C#

Type	Supported
Single	✅
Multilevel	✅
Hierarchical	✅
Multiple (class)	❌
Multiple (interface)	✅

Constructor Execution Order

Parent Constructor

↓

Child Constructor

CLR ensures base initialization first.

base Keyword

```
class Student : Person
{
    public Student(string name) : base(name)
  {
  }
}
```

Calls parent constructor.

5 METHOD OVERRIDING (Runtime Polymorphism)

Parent defines virtual method.

```
class Animal
{
    public virtual void Sound()
  {
    Console.WriteLine("Animal sound");
  }
}
```

Child overrides:

```
class Dog : Animal
{
    public override void Sound()
  {
    Console.WriteLine("Bark");
  }
}
```

Runtime Behavior

Animal a = new Dog();
a.Sound();
Output:
Bark

CLR decides method at runtime using **Virtual Method Table (VTable)**.

6 INTERFACES

Interface = contract.

It defines **WHAT must be done**, not HOW.

Real-Life Example

USB Port Standard:

- Any device must follow USB rules.

Interface Syntax

```
interface IPrintable
{
    void Print();
}
Implementation:
class Report : IPrintable
{
    public void Print()
   {
    Console.WriteLine("Printing report");
   }
}
```

Why Interfaces?

Problem without interface

```
class Payment
{
    CreditCard card = new CreditCard();
}
```

Tightly coupled.

With Interface

```
interface IPayment
{
        void Pay();
}
```

Now:

- CreditCard
- UPI
- NetBanking

all interchangeable.

Multiple Interface Implementation

```
class Machine : IPrint, IScan
{
}
```

Allowed because no ambiguity.

7 CLASS vs STRUCT vs RECORD vs TUPLE

CLASS

Reference Type.

Stored in Heap.

```
class Employee
{
        public string Name;
}
```

Behavior

```
Employee e1 = new Employee();
Employee e2 = e1;
e2.Name = "Changed";
```

Both change.

Because same reference.

When to Use Class

✅ Large objects
✅ Mutable data
✅ Business logic
Examples:

- Student
- Order
- Account

STRUCT

Value Type.
Stored in Stack.

```
struct Point
{
    public int X;
    public int Y;
}
```

Copy created during assignment.

```
Point p1;
Point p2 = p1;
```

Independent copies.

When to Use Struct

✅ Small data
✅ Lightweight objects
✅ Performance critical
Examples:

- Coordinates
- RGB color
- Date components

RECORD

Designed for **immutable data models**.

```
record Student(string Name, int Age);
```

Special Feature: Value Equality

```
var s1 = new Student("Naveen",23);
var s2 = new Student("Naveen",23);
Console.WriteLine(s1 == s2);
```

Output:

True

Class would return False.

Why Records?

Used heavily in:

- APIs
- DTOs
- Microservices
- Data transfer

TUPLE

Temporary grouped values.

```
var data = (Name:"Naveen", Age:23);
```

Access:

```
Console.WriteLine(data.Name);
```

Tuple Use Case

Returning multiple values:

```
(string,int) GetStudent()
{
    return ("Naveen",23);
}
```

No need for class creation.

COMPARISON TABLE

Feature	Class	Struct	Record	Tuple
Type	Reference	Value	Reference	Value
Memory	Heap	Stack	Heap	Stack

Mutable	Yes	Yes	Usually No	Yes
Equality	Reference	Value	Value	Value
Use	Business objects	Small data	DTO models	Temporary data

REAL WORLD EXAMPLE (COMBINED)

College API Model:

record StudentDTO(string Name,int Age);

Graphics Engine:

struct Point { int X; int Y; }

Business Logic:

class StudentService { }

Temporary Calculation:

(var total,var avg) = Calculate();

COMMON CONFUSIONS

❌ **Struct for large objects**

Bad performance due to copying.

❌ **Using class for simple coordinates**

Unnecessary heap allocation.

❌ **Interface with implementation fields**

Interfaces cannot have instance fields.

- 📦 **ARRAYS IN C#**

✅ What is an Array?

An **array** is a collection of elements of the **same datatype stored in continuous memory locations**.

Real-life analogy

Think of a **student exam hall seating row**:

Seat1--> Seat2--> Seat3--> Seat4--> Seat5

- Same type → Students
- Ordered positions → Index numbers
- Fixed seats → Fixed size

- **Why Arrays?**

Without arrays:

int s1 = 80;

int s2 = 85;

int s3 = 90;

Messy and non-scalable.

With array:

int[] marks = {80,85,90};

Array Declaration

datatype[] arrayName;

Example:

int[] numbers;

Array Initialization

Method 1

int[] numbers = new int[5];

Creates 5 elements (default value = 0).

Method 2

int[] numbers = {10,20,30,40};

Memory Representation

Arrays are stored **contiguously in heap memory**.

Index: 0 1 2 3

Value: 10 20 30 40

Access:

Console.WriteLine(numbers[0]);

Important Properties

numbers.Length

Iterating Arrays

for loop

```
for(int i=0;i<numbers.Length;i++)
{
    Console.WriteLine(numbers[i]);
}
```

foreach loop (recommended)

```
foreach(int n in numbers)
{
    Console.WriteLine(n);
}
```

Cleaner and safer.

Types of Arrays

1 Single Dimensional Array

```
int[] arr = {1,2,3};
```

2 Multi-Dimensional Array (Matrix)

```
int[,] matrix =
{
    {1,2},
    {3,4}
};
```

Access:

```
matrix[1,0];
```

Used in:

- Image processing
- Game boards
- Mathematical matrices

- **3️⃣ Jagged Array (Array of Arrays)**

  Rows can have different sizes.

```
int[][] jagged = new int[2][];
jagged[0] = new int[]{1,2,3};
jagged[1] = new int[]{4,5};
```

  Real-life:

Different students having different subject counts.

Array Limitations

❌ Fixed size

❌ Cannot grow dynamically

❌ Insert/delete costly

Solution → **Collections** (later section).

⚡DELEGATES, LAMBDAS & EVENTS

This is where C# becomes powerful.

These concepts allow:

✅ Method passing

✅ Callbacks

✅ Event-driven programming

✅ Loose coupling

🔹 Delegates

A **delegate** is a type-safe reference to a method.

Think:

Variable that stores a function.

Real-Life Analogy

Remote control:

- Remote button = delegate
- TV action = method
- Remote doesn't care which TV performs action.

Delegate Syntax

```
delegate void Message();
```

Example

```
class Program
{
    delegate void Message();
    static void Hello()
    {
    Console.WriteLine("Hello");
```

```
    }

        static void Main()

    {

        Message msg = Hello;

        msg();

    }

}
```

Why Delegates?

Allows passing methods as parameters.

```
void Execute(Action action)

{

        action();

}
```

Built-in Delegates

Action (no return)

```
Action greet = () => Console.WriteLine("Hi");
```

Func (returns value)

```
Func<int,int> square = x => x*x;
```

Predicate (returns bool)

```
Predicate<int> check = x => x > 10;
```

🔹 Lambda Expressions

Short anonymous functions.

Instead of:

```
int Square(int x)
{
    return x*x;
}
```

Write:

```
x => x * x
```

Lambda Structure

```
(parameters) => expression
```

Example

Func<int,int> cube = x => x*x*x;

Real Usage (LINQ)

var result = numbers.Where(x => x > 50);

Lambda defines filtering rule.

🔹 Events

Events are built using delegates.

Event = notification mechanism between objects.

Real-Life Example

Doorbell system:

Visitor presses bell

↓

House owner notified

Publisher → Subscriber.

Event Example

class Alarm

```
{
    public event Action Ring;
    public void Trigger()
    {
    Ring?.Invoke();
    }
}
```

Subscriber:

```
Alarm a = new Alarm();
a.Ring += () => Console.WriteLine("Wake up!");
a.Trigger();
```

Output:

Wake up!

Why Events?

Used everywhere:

- Button click

    - File download completed
    - Payment success
    - Email received
-

🧺 COLLECTIONS

Collections solve array limitations.

Arrays = fixed container

Collections = expandable smart containers.

Namespace

using System.Collections.Generic;

Types of Collections

- **List<T>**

Dynamic array.

List<int> numbers = new List<int>();

numbers.Add(10);

numbers.Add(20);

Automatically resizes.

Internal Working

List internally uses array and doubles capacity when full.

Capacity: 4 → 8 → 16 → 32

Useful Methods

numbers.Remove(10);

numbers.Contains(20);

numbers.Count;

- **Dictionary<TKey,TValue>**

Stores key-value pairs.

Real-life:

RollNo → StudentName

```
Dictionary<int,string> students =
    new Dictionary<int,string>();
students.Add(1,"Naveen");
```

Access:

students[1];

Fast lookup using hashing.

- **Stack (LIFO)**

Last In First Out.

Example:

Undo operation.

```
Stack<int> stack = new Stack<int>();

stack.Push(10);

stack.Pop();
```

- **Queue (FIFO)**

First In First Out.

Example:

Ticket counter.

```
Queue<string> q = new Queue<string>();

q.Enqueue("A");

q.Dequeue();
```

- **HashSet**

Stores unique values.

```
HashSet<int> set = new HashSet<int>();

set.Add(1);

set.Add(1); // ignored
```

ARRAY vs COLLECTION

Feature	Array	List
Size	Fixed	Dynamic
Speed	Faster	Slightly slower
Flexibility	Low	High
Memory	Efficient	More overhead

-

REAL WORLD COMBINED EXAMPLE

Student Notification System:

```
class StudentService

{

    public event Action<string> OnAdded;

    List<string> students = new();
```

```
    public void AddStudent(string name)
  {
    students.Add(name);
    OnAdded?.Invoke(name);
  }
}
```

Usage:

```
var service = new StudentService();
service.OnAdded += n =>
    Console.WriteLine($"{n} added");
service.AddStudent("Naveen");
```

COMMON CONFUSIONS

❌ Array vs List

Use List unless fixed size required.

❌ Delegate vs Method

Delegate stores method reference.

❌ Event vs Delegate

Event = protected delegate invocation.

- Arrays → store multiple data
- Collections → manage dynamic data efficiently
- Delegates & Events → enable communication between objects

These three together power **LINQ, ASP.NET, UI frameworks, async programming**, and almost all modern .NET applications.